MACHINE LEARNING — LAB MANUAL

Lab 3: Decision Tree Classification — Training, Evaluation & Overfitting

A 3-Hour Hands-On Lab — BS Computer Science, 5th Semester

Duration: 3 Hours (independent, hands-on)

Prerequisites: Lab 1 (Python/NumPy/Pandas/Matplotlib, Kaggle & Colab basics); Lab 2 (data cleaning & EDA on Telco Churn); Decision Tree theory from the AI course

Tools Required: Python 3.x, Pandas, NumPy, scikit-learn, Matplotlib/Seaborn, Google Colab

Includes: Task-driven model training, evaluation, an overfitting experiment, feature importance, and a graded rubric

Learning Outcomes

This lab is NOT a re-introduction to Python, Pandas, or Decision Tree theory (link1, link2)— you already know these. The goal is to practice the end-to-end workflow of turning an ML-ready dataset into a properly evaluated classifier, and to build real intuition for overfitting. By the end of this lab you should be able to independently:

- Prepare features and a target variable for a scikit-learn classifier
- Split data into training and test sets correctly
- Train a baseline Decision Tree classifier
- Evaluate a classifier using accuracy, precision, recall, F1-score, and a confusion matrix
- Compare training vs. test performance to detect overfitting
- Control model complexity using max_depth and interpret the resulting trade-off
- Select and justify a final model based on evidence, not just top accuracy
- Read and interpret feature importances in a business context

Module	Focus Area
1	Problem Definition & Dataset Verification (0–15 min)
2	Feature & Target Preparation (15–35 min)
3	Train/Test Split (15–35 min)
4	Baseline Decision Tree (35–60 min)
5	Model Evaluation (60–90 min)
6	Overfitting Investigation (90–130 min)
7	Model Selection, Feature Importance & Interpretation (130–150 min)
8	Conclusion & Next Steps (150–180 min)

Scenario

You now have a clean, ML-ready version of the Telco Customer Churn dataset from Lab 2. Today's job is to turn it into an actual classifier: prepare it for scikit-learn, train a Decision Tree, evaluate it properly (not just with accuracy), and understand how tree depth trades off underfitting against overfitting. This is hands-on the whole way through — you write the code; this manual only tells you what to do and what to think about.

Before You Start

- Open your Lab 2 notebook/output and locate clean_churn.csv (or the equivalent cleaned dataframe).
- Create a new Colab notebook: lab3_<yourname>_churn_dt.ipynb, with the student-info block and Markdown headings described in Submission Requirements.
- You will submit only this one notebook — make sure every section asked for is present before you finish.

Part 1 — Problem Definition & Dataset Verification

Before training anything, restate the problem in your own words and confirm the cleaned dataset from Lab 2 is intact — a model is only as trustworthy as the data feeding it.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
pd.read_csv(...)	Load the cleaned dataset saved in Lab 2
df.shape / df.info()	Confirm rows, columns, and dtypes are as expected
df['Churn'].value_counts()	Reconfirm the target and its class balance

Tasks

1. Load clean_churn.csv (or re-run your Lab 2 cleaning). Verify its shape, column dtypes, and that there are no missing values remaining.
2. In 1–2 sentences, restate the ML problem: what exactly are you predicting, and why does it matter to the business?

Part 2 — Feature & Target Preparation

scikit-learn models require numeric input. Separate your target from your features, then convert any remaining categorical columns into a numeric form a Decision Tree can use.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
pd.get_dummies(df, columns=[...])	One-hot encode categorical columns
df[col].map({...})	Manually encode a binary column (e.g. Churn -> 0/1)
df.drop(columns=[...])	Remove columns that shouldn't be model inputs
X.isnull().sum() / X.dtypes	Confirm X is fully numeric with no missing values

Tasks

3. Define y as the Churn column, encoded as 0/1. Define X as the remaining relevant columns — drop any identifier column still present.
4. Convert every remaining categorical column in X into numeric form. Briefly justify your choice of encoding method (e.g. one-hot vs. manual mapping).
5. Confirm X contains only numeric columns and has no missing values before moving on.

Part 3 — Train/Test Split

A model must be evaluated on data it has never seen. Hold out a test set before training anything.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
train_test_split(X, y, test_size=..., random_state=..., stratify=y)	Splits data into training and test sets

Tasks

6. Split X and y into training and test sets (e.g. an 80/20 split). Why is stratify=y a sensible choice, given what you found about the target's class balance in Lab 2?
7. Report the shape of X_train, X_test, y_train, and y_test.

Part 4 — Baseline Decision Tree

Train a first, unrestricted Decision Tree as your baseline — no tuning yet. This gives you something concrete to evaluate and improve on.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
DecisionTreeClassifier(random_state=...)	Creates a Decision Tree classifier
.fit(X_train, y_train)	Trains the model
.predict(X)	Generates predictions for a given feature set
.get_depth()	Reports how deep the fitted tree actually grew

Tasks

8. Train a DecisionTreeClassifier on the training data with default settings (only random_state fixed). Generate predictions on both the training and test sets.
9. What depth did scikit-learn actually grow this tree to on its own? Does that surprise you?

Part 5 — Model Evaluation

Accuracy alone can be misleading — especially given the class imbalance you found in Lab 2. Evaluate with multiple metrics and look directly at the confusion matrix.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
accuracy_score(y_true, y_pred)	Overall proportion of correct predictions
precision_score / recall_score / f1_score	Class-specific performance metrics
confusion_matrix(y_true, y_pred)	Breakdown of correct/incorrect predictions by class
ConfusionMatrixDisplay / classification_report	Visualize or summarize all metrics at once

Tasks

10. Compute accuracy, precision, recall, and F1-score on the test set. Report them in a small table.
11. Plot the confusion matrix for the test set. How many actual churners did the model correctly catch, and how many did it miss?
12. Given Lab 2's finding about class imbalance, which metric(s) do you trust most for this problem, and why?

Part 6 — Overfitting Investigation

A model that performs far better on training data than on test data has overfit — it has memorized noise rather than learned a generalizable pattern. Investigate this directly by comparing performance and by controlling tree depth with max_depth.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
accuracy_score(y_train, model.predict(X_train))	Training-set performance, for comparison against test performance
DecisionTreeClassifier(max_depth=k)	Limits how deep the tree is allowed to grow
for k in [2,3,4,5,6,8,10,None]: ...	Loop to train/evaluate multiple depths
plt.plot(...)	Plot train vs. test accuracy against max_depth

Tasks

13. Compare your baseline tree's accuracy on the training set vs. the test set. Is there a large gap? What does that suggest about the model?
14. Train several Decision Trees at different max_depth values (e.g. 2, 3, 4, 5, 6, 8, 10, and unrestricted). Record train accuracy and test accuracy for each in a results table.
15. Plot train accuracy and test accuracy against max_depth on the same chart. At roughly what depth does the model start to overfit? Explain how the chart shows this.

Part 7 — Model Selection, Feature Importance & Interpretation

Pick the depth that best balances training and test performance — not simply the one with the highest training accuracy — then look inside the model to see what it actually learned.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
model.feature_importances_	Relative importance scikit-learn assigned each feature
pd.Series(importances, index=X.columns).sort_values()	Rank features by importance
plt.barh(...)	Visualize feature importances

Tasks

16. Based on your Part 6 table/plot, select a max_depth you consider the best model. Justify your choice in 2–3 sentences — it should not simply be the depth with the highest training accuracy.
17. Retrain a Decision Tree at your chosen depth and report its full evaluation metrics (as in Part 5) for comparison against the baseline.
18. Retrain a Decision Tree with "**entropy**" as the splitting criterion rather than the default ("**gini**") and report its full evaluation metrics (as in Part 5) for comparison against the baseline.
19. Extract and visualize feature_importances_ for your chosen model. Which 3–5 features matter most?
20. In 2–3 sentences, connect the top features to what you already found in Lab 2's EDA — do they agree with the patterns you saw then?

Part 8 — Conclusion & Next Steps

Step back from the code and summarize what you built and learned.

Tasks

21. Write a short conclusion (4–6 sentences): which model did you land on, how well does it perform, and what are its main limitations?
22. What would you try next if you had more time (e.g. a different algorithm, more feature engineering, handling class imbalance)? Describe it — do not implement it here.
23. Implement **ID3** from **scratch** on the Play Badminton dataset from class. No built-in decision tree libraries — write your own entropy and information gain functions. Print the gain values for each attribute at every split.

Submission Requirements

Submit exactly ONE Google Colab notebook. At the very top, include a student-info block with: Name, Student ID, Section, GitHub Profile, Kaggle Profile, Dataset, and Dataset Source.

Your Notebook Must Contain These Sections

- 1. Problem Definition 2. Dataset Verification 3. Feature/Target Preparation 4. Train/Test Split

- 5. Baseline Decision Tree 6. Model Evaluation 7. Overfitting Experiment 8. Model Comparison
- 9. Feature Importance 10. Interpretation 11. Conclusion

Every table, plot, and metric must be accompanied by a short written interpretation — code with no discussion will not be accepted. End the notebook with a short Submission Checklist confirming every section above is present.

Penalties

- Evaluating with accuracy only (no precision/recall/F1/confusion matrix): marks deducted in Part 5.
- No explicit train-vs-test comparison, or no multi-depth experiment: marks deducted in Part 6.
- Selecting a depth with no justification, or choosing purely by training accuracy: marks deducted in Part 7.
- Feature importances reported with no interpretation: no credit for that section.
- Using Random Forest, GridSearchCV, or RandomizedSearchCV in this lab (not requested): marks deducted — save it for a later lab.